Bug and Defect History Report

Deliverable 3, Campus Ride-Pooling and Split-Fare System

Prepared by Khushi Ranawat Jira CS455-135

Prithviraj Ghosh (230794) Mainak Sarkar (230619) Khushi Ranawat (230562) Mahir Jain (230618) Chaitanya Nitawe (230315)

Executive summary. This report records 26 distinct, resolved application defects: 16 from the D3 security, reliability, and database audit, and 10 product defects found during the D1 code review recorded in CS455-86. Overlapping observations are consolidated so the same underlying issue is not counted twice. Each entry summarizes system impact, cause, correction, and the available regression, pull-request, and Jira evidence.

1. Scope and evidence

The report covers defects with a concrete failure mode and a corresponding code fix. It includes authentication and authorization, realtime workflows, database reliability, request integrity, production configuration, and product-rule defects found by review of the D1 requirements. The first 16 entries cover the D3 audit; entries 17–26 cover the ten product defects fixed in PR #67 and tracked by CS455-86. Duplicate observations are consolidated into one defect; supporting test details are retained as evidence rather than counted as separate bugs.

2. Defect summary

No.	Area	Resolved defect
1	Realtime security	Credentialed cross-origin Socket.IO access
2	Session security	Realtime authentication bypassed inactivity expiry
3	Database security	PostgreSQL TLS certificate verification disabled
4	Migration reliability	Migration referenced `rides.locked_at` before adding it
5	Abuse prevention	Client-controlled forwarding headers bypassed IP rate limits
6	Realtime robustness	Malformed cookies could throw during websocket authentication
7	Migration reliability	Existing request tables could lack `updated_at`
8	Credential integrity	Password-reset token consumption and credential changes were not atomic
9	Session correctness	Active realtime use did not refresh session activity
10	Authorization	Realtime sockets bypassed profile and warning-acknowledgement gates
11	Production configuration	Missing production `APP_URL` was not rejected at startup
12	Session security	Production realtime accepted the development cookie name
13	Session security	HTTP refresh could revive an inactivity-expired session
14	Database correctness	Existing cursor indexes were not upgraded by `IF NOT EXISTS`
15	Request reliability	Untyped timestamp parameters caused PostgreSQL 500 responses
16	Request integrity	Reusing an idempotency key for different request details replayed the wrong request
17	Ride matching	One rider could hold overlapping seats or offer overlapping rides (P-08)
18	Suspension enforcement	A suspended rider could continue posting in pool chats
19	Complaint retention	Pool chat deletion could remove messages cited by an unresolved complaint
20	Admin security	Chat-report resolution lacked audit and request protections
21	Complaint eligibility	A rider who left before the trip could still file trip complaints
22	Ratings and history	Completed-trip counts and rating eligibility included no-shows or leavers
23	Ride editing	Edited rides did not enforce the three-hour departure-window limit
24	Account integrity	Account deletion could orphan active seats or waiting requests
25	Request notifications	Itinerary reconfirmation silently returned a request to Pending
26	Trip lifecycle	Scheduled ride cancellation and start did not consistently respect the departure-window end

3. Resolved defects

3.1 Credentialed cross-origin Socket.IO access

Impact. The Socket.IO server reflected arbitrary browser origins while allowing credentials. A malicious origin could attempt to establish a socket using a victim's authenticated browser session and access ride-chat or notification events available to that victim. **Cause.** The CORS configuration used a permissive origin reflector with credentialed access. **Fix.** Socket.IO now validates the browser origin during the connection handshake with the server's `allowRequest` callback in `server.mjs`. A request carrying an `Origin` header is accepted only when it exactly matches the configured application origin; an untrusted origin is rejected before session authentication. The CORS allow-list remains configured for HTTP long-polling, but is not treated as WebSocket handshake protection. **Evidence.** The WebSocket handshake regression in `apps/web/test/server-security.test.ts` rejects an untrusted Origin. The handshake enforcement uses `allowRequest` in `server.mjs`; it was added in PR #94, not by the CORS allow-list in PR #91. Jira CS455-136.

3.2 Realtime authentication bypassed inactivity expiry

Impact. A remembered session inactive beyond the seven-day policy could be rejected by HTTP routes but still authenticate a Socket.IO connection. **Cause.** Realtime session lookup checked absolute expiry but omitted the inactivity predicate applied by HTTP authentication. **Fix.** Realtime authentication and revalidation now enforce the same seven-day inactivity bound as HTTP session validation. **Evidence.** `apps/web/test/server-security.test.ts` verifies that a Socket.IO connection is rejected for a session outside the seven-day inactivity window. Code fix in PR #91; regression test in PR #94; Jira CS455-137.

3.3 PostgreSQL TLS certificate verification was disabled

Impact. External PostgreSQL connections were encrypted but did not verify the server certificate, allowing an intercepted connection to use an untrusted certificate. **Cause.** Render-style connection configuration set `rejectUnauthorized: false`. **Fix.** Certificate verification is enabled consistently in the application pool, custom server, migration script, and administrative role script. **Evidence.** `apps/web/src/lib/db.test.ts` verifies that the Render PostgreSQL URL path enables certificate verification. PR #91; Jira CS455-138.

3.4 Migration referenced `rides.locked_at` before adding it

Impact. Upgrading an older database could fail while creating a partial index that referenced `locked_at`, blocking startup. **Cause.** The index was declared before the compatibility `ALTER TABLE` statement. **Fix.** The compatibility column addition now runs before every index or predicate that references the column; the duplicate later alteration was removed. **Evidence.** The migration ordering and dependency were reviewed in PR #91; no automated legacy-schema migration test exists. Jira CS455-139.

3.5 Client-controlled forwarding headers bypassed IP rate limits

Impact. Varying a trusted-looking forwarding-header value could create a new rate-limit key for repeated requests, weakening throttling on authentication, registration, chatbot, and ride mutation endpoints. **Cause.** IP extraction trusted the first client-supplied `X-Forwarded-For` value and a separate `X-Real-IP` path. **Fix.** The separate client-controllable `X-Real-IP` trust path was removed. Extraction uses the final proxy-appended `X-Forwarded-For` hop and bounds the key value. The overlapping header observations are counted once. **Evidence.** `apps/web/src/lib/auth.test.ts` verifies final-hop `X-Forwarded-For` selection and confirms that `X-Real-IP` does not override it. PR #91; Jira CS455-140.

3.6 Malformed cookies could throw during websocket authentication

Impact. A malformed percent-encoded cookie could throw during websocket authentication instead of following the normal unauthenticated rejection path. **Cause.** Cookie decoding did not handle malformed percent-encoding. **Fix.** Decoding is wrapped in a narrow error handler. The malformed cookie is treated as empty and authentication fails closed without an uncaught parser exception. **Evidence.** `apps/web/test/server-security.test.ts` supplies a malformed percent-encoded cookie, verifies authentication rejection, and confirms that a subsequent valid connection still succeeds. Code fix in PR #91; regression test in PR #94; Jira CS455-141.

3.7 Existing request tables could lack `updated_at`

Impact. An older `ride_join_requests` table without `updated_at` could fail when the schema bootstrap referenced the missing column. **Cause.** The compatibility migration did not add `updated_at` before subsequent backfill statements used it. **Fix.** The schema adds the column when absent, backfills it from available timestamps, and applies its default and not-null constraint before later references. **Evidence.** The compatibility migration and statement ordering were reviewed in PR #91; no automated legacy-schema migration test exists. Jira CS455-142.

3.8 Password-reset operations were not atomic

Impact. A failure after token consumption but before updating the password or revoking sessions could leave a reset partially completed. **Cause.** Token consumption, password update, and session deletion were separate database operations. **Fix.** These operations now run in one transaction and commit or roll back together. Password hashing runs before acquiring the database connection, limiting transaction and row-lock duration. **Evidence.** `apps/web/src/app/api/auth/reset/route.test.ts` covers commit and rollback behavior; PR #94 adds the failure-injection integration test in `apps/web/test/integration/auth/reset-failure.int.test.ts`. PRs #91 and #94; Jira CS455-143.

3.9 Active realtime use did not refresh session activity

Impact. A user actively using realtime features could have a session expire under the inactivity policy. **Cause.** Successful realtime authentication checked the session but did not refresh its activity timestamp. **Fix.** Authenticated realtime activity conditionally updates `last_activity_at`, with a five-minute write throttle. **Evidence.** The server-side refresh and five-minute throttle were reviewed in PR #91; no automated test covers the realtime refresh/throttle path. Jira CS455-144.

3.10 Realtime sockets bypassed account eligibility gates

Impact. Users with incomplete profiles or unacknowledged warnings could be blocked from protected HTTP workflows but still authenticate realtime access. **Cause.** Socket authentication omitted profile-completion and warning-acknowledgement checks. **Fix.** Realtime authentication and revalidation now enforce the same eligibility gates as HTTP requests, with the same admin exception. **Evidence.** The realtime profile and warning predicates were reviewed in PR #91; no automated realtime eligibility test exists. Jira CS455-145.

3.11 Missing production `APP_URL` was not rejected at startup

Impact. Production could start with an empty Socket.IO origin allow-list, silently preventing browser realtime connections. **Cause.** The server fell back to an empty origin when `APP_URL` was not configured outside development. **Fix.** The server now fails fast in production when `APP_URL` is absent or blank. **Evidence.** The fail-fast startup validation was reviewed in PR #91; no automated test covers the missing-production-`APP_URL` startup case. Jira CS455-146.

3.12 Production realtime accepted the development cookie name

Impact. Production Socket.IO authentication accepted the development session-cookie name as an alternative to the host-prefixed production cookie. **Cause.** Realtime authentication accepted either cookie name regardless of environment. **Fix.** The cookie name is selected by environment: development uses `crp_session`; production accepts only `__Host-crp_session`. **Evidence.** `apps/web/src/lib/auth.test.ts` verifies the HTTP production `__Host-` cookie policy only. The realtime cookie-name selection was reviewed in PR #91; no automated test covers rejection of the development cookie name by realtime authentication. Jira CS455-147.

3.13 HTTP authentication could revive an inactivity-expired session

Impact. An idle-expired remembered session could be rejected during lookup and then refreshed by the same request, allowing later authentication with the stale cookie. **Cause.** The refresh update omitted the expiry and seven-day inactivity predicates used by active-session lookup. **Fix.** Refresh now requires both absolute expiry and recent activity, so only an already-valid session can be updated. **Evidence.** `apps/web/src/lib/auth.test.ts` checks the seven-day inactivity condition, absolute expiry, and the refresh predicate. PR #91; Jira CS455-148.

3.14 Existing cursor indexes were not upgraded by `IF NOT EXISTS`

Impact. Existing databases retained older same-named indexes, so cursor ordering continued to use outdated index definitions. **Cause.** PostgreSQL skips creation when an index of the same name exists; it does not replace its definition. **Fix.** The affected notification and complaint-analysis cursor indexes are explicitly dropped and

recreated with the required ordering columns. **Evidence.** The explicit drop-and-recreate statements were reviewed in the schema change in PR #91; no automated cursor-index migration test exists. Jira CS455-149.

3.15 Untyped timestamp parameters caused PostgreSQL 500 responses

Impact. Seat-request submission and the payment-reminder scheduler could fail against PostgreSQL with a parameter type/operator error and return HTTP 500. **Cause.** JavaScript `Date` parameters were used in timestamp/interval expressions without an explicit PostgreSQL timestamp type. **Fix.** The affected SQL parameters now use explicit `timestamptz` casts. **Evidence.** Four focused regression tests are in `apps/web/test/integration/bugs/timestamp-parameters.int.test.ts`. PR #84; Jira CS455-133 (Bug).

3.16 Idempotency-key reuse replayed a different request

Impact. Reusing a key for another ride or pickup point returned the original request as though it were a valid retry. A rider could be told a request already existed even though nothing was created for the intended ride, or a changed pickup point could be ignored. **Cause.** The idempotency check treated the key alone as sufficient to identify a replay and did not require the ride and pickup payload to match. **Fix.** A key now replays only the same logical request. Reuse with different ride or pickup details returns HTTP 409 with `idempotency_key_reused`; the client discards the stale key. **Evidence.** Five regression cases are in `apps/web/test/integration/bugs/idempotency-key-reuse.int.test.ts`. PR #87; Jira CS455-134 (Bug).

4. Product defects found during D1 review (CS455-86)

The following ten additional product defects were found by reviewing the D1 requirements and were fixed in PR #67. They are included because the D3 plan counts review findings as well as the D3 audit findings. Each is separately tracked as a Jira Bug and related to the CS455-86 tracking task.

4.1 One rider could hold overlapping seats or offer overlapping rides (P-08)

Impact. One person could participate in two pools at once: holding a seat on an unfinished ride while requesting or accepting another ride within two hours, or offering an overlapping ride. **Cause.** The travel-conflict rule was implemented inconsistently across request, acceptance, offer, edit, and leave operations. **Fix.** PR #67 centralizes the rule in `travelConflict()` and applies it across all relevant operations. Offering or moving a ride also withdraws the owner's own waiting requests that conflict with the new ride. **Evidence.** PR #67; regression coverage in `apps/web/src/lib/join-requests.test.ts` and `apps/web/src/app/api/rides/route.test.ts`; Jira CS455-150.

4.2 A suspended rider could continue posting in pool chats

Impact. Suspension did not consistently prevent a rider from posting chat messages through both the realtime socket and HTTP send paths. **Cause.** The send paths did not share a single suspension enforcement decision. **Fix.** PR #67 routes both paths through `chatSendRefusal()` and a shared suspension rule, while preserving the exemption for the owner of an existing ride. **Evidence.** PR #67; tests in `apps/web/src/lib/ride-chat-realtime.test.ts`, `apps/web/src/lib/ride-chat.test.ts`, and `apps/web/src/lib/suspension-rule.test.ts`; Jira CS455-151.

4.3 Pool chat deletion could remove messages cited by an unresolved complaint

Impact. A pool chat could be deleted after 30 days while an unresolved complaint still cited one of its messages, removing the links needed for complaint review. **Cause.** Chat-retention checks did not consistently account for unresolved complaints that cited individual messages. **Fix.** PR #67 centralizes the retention predicate in `CHAT_KEPT_FOR_COMPLAINT_SQL` and applies it to both scheduled maintenance and lazy deletion. **Evidence.** PR #67; tests in `apps/web/src/lib/ride-chat.test.ts` and `apps/web/src/lib/ride-chat-realtime.test.ts`; Jira CS455-152.

4.4 Admin chat-report resolution lacked audit and request protections

Impact. The admin chat-report endpoint could change complaint state without a corresponding audit record, same-origin validation, or rate limiting, and it remained accessible to the development test admin. **Cause.** The route did not use the shared protected admin-action workflow. **Fix.** PR #67 applies `requireAdminApi` and runs the mutation through `withAdminAction`, committing the state change and audit entry together. **Evidence.** PR #67; tests in `apps/web/src/app/api/admin/chat-reports/route.test.ts`; Jira CS455-153.

4.5 A rider who left before the trip could still file trip complaints

Impact. A rider who left before the trip and owed nothing could still appear in trip complaint eligibility and submit a trip complaint. **Cause.** Complaint eligibility did not distinguish a rider who left before the trip from a late leaver who still owed a locked share. **Fix.** PR #67 adds the shared `leftBeforeTrip()` rule. A rider who left before the trip cannot file or receive a trip complaint, while late leavers who still owe their share remain eligible. **Evidence.** PR #67; tests in `apps/web/src/lib/complaint-rules.test.ts` and `apps/web/src/lib/complaints.test.ts`; Jira CS455-154.

4.6 Completed-trip counts and rating eligibility included no-shows or leavers

Impact. Public completed-trip counts included no-shows (and profile counts included riders who left); ride history could show a Rate link to a rider who had left. **Cause.** History and profile aggregation did not consistently exclude no-show or left seats, and rating eligibility did not include the leave condition. **Fix.** PR #67 excludes left and no-show seats from the relevant counts and makes rating unavailable to someone who left. **Evidence.** PR #67; tests in `apps/web/src/lib/ride-view.test.ts` and `apps/web/src/lib/profile-data.test.ts`; Jira CS455-155.

4.7 Editing a ride did not enforce the three-hour maximum departure window

Impact. An owner could edit a ride to a departure window longer than the three-hour maximum applied when creating a ride. **Cause.** The edit path did not reuse the validation applied to ride offers. **Fix.** PR #67 applies the same maximum-window rule when editing and returns HTTP 400 with `invalid_departure` for an invalid window. **Evidence.** PR #67; tests in `apps/web/src/lib/join-requests.test.ts` and `apps/web/src/app/api/rides/[id]/route.test.ts`; Jira CS455-156.

4.8 Account deletion could orphan active seats or waiting requests

Impact. Deleting an account could anonymize a rider who still held a seat on an active or upcoming ride, or had a waiting request, leaving ride state without an account able to manage it. **Cause.** Account deletion did not block these outstanding ride and request relationships before anonymization. **Fix.** PR #67 refuses deletion with HTTP 409 and `active_rides` while an active/upcoming seat or waiting request remains. Password hashing is deferred until deletion can proceed. **Evidence.** PR #67; tests in `apps/web/src/app/api/profile/delete/route.test.ts`; Jira CS455-157.

4.9 Itinerary reconfirmation silently returned a request to Pending

Impact. After itinerary drift, a rider's reconfirmation could return a request to Pending without notifying the ride owner that the request needed attention again. **Cause.** The reconfirmation path changed request state without sending the owner notification used by the related decline path. **Fix.** PR #67 sends the owner a `join_request` notification when reconfirmation returns the request to Pending. **Evidence.** PR #67; tests in `apps/web/src/lib/join-requests.test.ts`; Jira CS455-158.

4.10 Scheduled ride cancellation and start did not consistently respect the departure-window end

Impact. An owner could not cancel a Scheduled ride after its departure time had passed even though pickup had not started; start and cancel operations did not share a clear end boundary. **Cause.** The Scheduled-to-start and cancellation paths did not consistently use the end of the departure window. **Fix.** PR #67 permits cancellation and start until the departure window ends, refuses both after that point with `window_over`, and does not cut off a trip already under way. **Evidence.** PR #67; tests in `apps/web/src/lib/trip-rules.test.ts`, `apps/web/src/lib/trip.test.ts`, and `apps/web/src/components/rides/RideManagementControls.test.tsx`; Jira CS455-159.